Kernels, Padding, Shape Debugging, Image Classification

Sattam Altwaim
satttam.tuuaim@kaust.edu.sa



جامعة الملك عبدالله
للعلوم والتقنية
King Abdullah University of
Science and Technology



KAUST Academy & Mawhiba
AI Olympiad Training (Beginners)
Week 4, Day 25, Self-study

13 October 2026

Table of Contents

1. Where This Fits
2. Learning Outcomes
3. Kernels in Depth
4. Padding and Stride in Depth
5. Shape Debugging
6. Image Classification End to End
7. Olympiad Practice
8. Check Yourself
9. Limitations and What Comes Next
10. Summary
11. References

From the Lecture

- **In the lectures:** Day 22 defined the kernel and the feature map, computed one window by hand, ran four hand-made kernels over a photo, showed the kernel as a cube over the colour channels, gave the output-shape formula for padding and stride, and wrote the CNN as a class with "print the shapes before you train". Day 23 stacked blocks into stages with a global-average-pooling head and added augmentation. The labs built `SceneCNN` and `VGGSmall` on Kaggle photos.
- **This deck adds:** a gallery of kernels and the rule their sums obey; the 1×1 kernel; receptive-field arithmetic; the three ways to pad and what the border loses; the floor in the formula, traced through a 150-pixel photo; every shape error PyTorch throws, with the message, the cause and the fix; `summary`, `LazyLinear` and global average pooling as tools; and the classification pipeline from a folder of photos to a submission file, with the numbers `ToTensor` and `Normalize` produce, the class-index trap and the test-time checklist.
- **How to use it:** every frame stands alone and every number was computed. Run each code frame in a notebook, break it on purpose, and read the message you get back. Answer the check at the end before you look at the answers.

Why Go Deeper

Where the simple version breaks

- Day 22's network was sized for 96×96 photos. Feed it the same Kaggle scenes at their native 150×150 and it stops with `mat1 and mat2 shapes cannot be multiplied (32x20736 and 9216x6)`. The fix is one number, but only for a reader who knows which number the message is naming.
- The formula rounds down. Three poolings take 150 to 75, to 37, to 18, not to 18.75: a head sized as "150 divided by 8" is wrong by thousands of features, and nothing in the lecture's table showed it.
- A model with validation accuracy 0.9 can hand in a worthless file: the training augmentation left on at test time, a shuffled test loader, or class indices in the wrong order. None of these crashes.

After this deck you can

size every layer of an image classifier from the formula, read any shape error back to the line that caused it, and run the pipeline from folder to submission with every scale, shape and index checked before training starts.

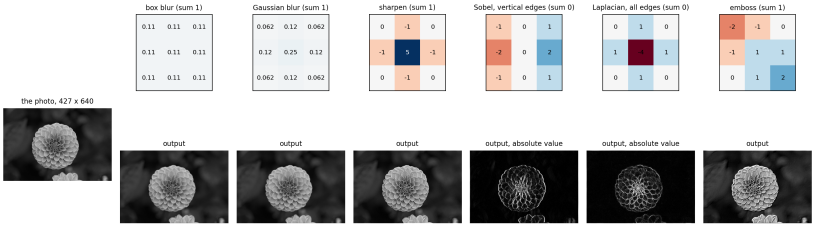
⇒ Learning Outcomes

By the end of this self-study deck, you will be able to:

- **Explain** what a kernel's numbers do from their sum, what a 1×1 kernel computes, and how far a stack of 3×3 kernels sees, with the arithmetic.
- **Choose** padding and stride on purpose and compute every output shape of a network, including the ones the floor changes.
- **Read** the shape errors of `Conv2d`, `Linear`, `MaxPool2d` and `CrossEntropyLoss` back to the line that caused them, and fix that line.
- **Use** a zero batch, `torchinfo.summary`, `LazyLinear` and global average pooling to make a model's shapes right before training.
- **Build** the image-classification pipeline from a folder of photos to a submission file: transforms with the right numbers, the split, the loaders, the model class, the loop, the confusion matrix.
- **Check** a run at test time: the evaluation transform, `eval` and `no_grad`, an unshuffled loader, the index-to-name mapping, the order of the rows.

Kernels: **What the Numbers Do**

A Gallery of Hand-Made Kernels



Source: scikit-learn's sample photograph `flower.jpg` (427 × 640, averaged to one grey channel), each kernel applied with `F.conv2d(x, k, padding=1)`; `scripts/day25_shape_figures.py`.

- Two families. The **blur and sharpen** kernels sum to 1 and give back a picture; the **edge** kernels sum to 0 and give back only the places where brightness changes.
- Sharpen is identity minus Laplacian: the pixel, plus how much it differs from its four neighbours. Emboss is a tilted edge kernel plus the identity, so it keeps the picture and lights one side of every edge.

The Sum Rule, With Numbers

Rule

On a **flat** patch of value v, a kernel outputs $v \cdot \sum_{u,v} K[u, v]$: the pixel value times the kernel's sum. Sum 1 keeps brightness; sum 0 outputs nothing until something changes.

Three patches

Flat patch, all 120: box blur $\frac{1}{9} \cdot 9 \cdot 120 = \mathbf{120}$; Sobel $120 \cdot (-1 - 2 - 1 + 0 + 1 + 2 + 1) = 120 \cdot 0 = \mathbf{0}$; sharpen $120 \cdot (5 - 4) = \mathbf{120}$. A vertical edge, columns $50, 50, 200$: Sobel $= (-1 - 2 - 1) \cdot 50 + (1 + 2 + 1) \cdot 200 = -200 + 800 = \mathbf{600}$. A bright dot, 200 among 100s: sharpen $= 5 \cdot 200 - 4 \cdot 100 = \mathbf{600}$, three times brighter than before; blur $= (200 + 8 \cdot 100)/9 = \mathbf{111}$, almost gone.

- So the sum tells you the family before you run anything, and the signs tell you the direction: Sobel's $-1, 0, 1$ columns respond to left-dark, right-bright.
- Learned kernels obey no such rule, yet after training on photos the first layer's kernels look like this gallery: edges at many angles, blobs, colour contrasts (Krizhevsky et al., 2012, Fig. 3). Gradient descent rediscovers the gallery.

Kernels on Channels, and the 1×1 Kernel

```
c = nn.Conv2d(3, 16, 3, padding=1)
print(c.weight.shape, sum(p.numel() for p in c.parameters()))   # (16, 3, 3, 3) 448
one = nn.Conv2d(64, 16, kernel_size=1)
print(one(torch.zeros(1, 64, 12, 12)).shape)      # (1, 16, 12, 12): same map size
print(sum(p.numel() for p in one.parameters()))   # 1040, the same as nn.Linear(64, 16)
```

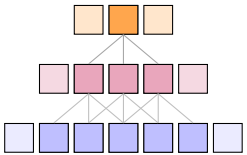
Reading a weight shape

`Conv2d(3, 16, 3)` has weight $(16, 3, 3, 3)$: **16 kernels**, each a $3 \times 3 \times 3$ cube over the 3 input channels. One output number is 27 products and a bias; $16 \cdot 27 + 16 = 448$ parameters. Read any `weight of size [a, b, k, k]` as a kernels, b input channels, a $k \times k$ window.

The 1×1 kernel

`Conv2d(64, 16, 1)` has weight $(16, 64, 1, 1)$: at every pixel it mixes the 64 channels into 16 and leaves the map size alone. It is `Linear(64, 16)` applied at each position, $64 \cdot 16 + 16 = 1{,}040$ parameters either way. U-Net's last layer `Conv2d(32, 1, 1)` (Day 24) and ResNet's channel changes are this kernel.

Stacking Kernels: The Receptive Field, With Arithmetic



Two 3×3 convolutions, drawn along one row: the top unit depends on five input pixels.

Recurrence

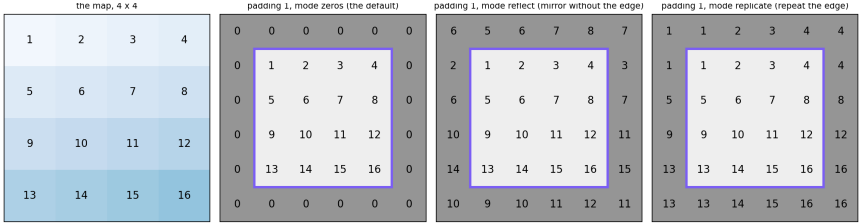
$r_\ell = r_{\ell-1} + (k-1) \cdot j_{\ell-1}, \quad j_\ell = j_{\ell-1} \cdot s$: a layer adds $k-1$ steps of the current **jump** j (pixels between neighbouring units); a stride multiplies the jump. Start at $r_0 = 1$, $j_0 = 1$.

layer	arithmetic	r	j
conv 3×3	$1 + 2 \cdot 1$	3	1
conv 3×3	$3 + 2 \cdot 1$	5	1
pool 2, stride 2	$5 + 1 \cdot 1$; jump $\times 2$	6	2
conv 3×3	$6 + 2 \cdot 2$	10	2
conv 3×3	$10 + 2 \cdot 2$	14	2
pool 2, stride 2	$14 + 1 \cdot 2$; jump $\times 2$	16	4
conv 3×3	$16 + 2 \cdot 4$	24	4

- Day 23's chain $3 \to 5 \to 10 \to 14 \to 24$, now with the rule that produces it; a third stage reaches 56. Two 3×3 layers see 5×5 with 18 weights instead of 25, and put a ReLU in between (Simonyan and Zisserman, 2015).
- A head on a map whose units see less than the object is guessing from parts: count r against the image size before you choose the depth.

Padding and Stride: **Every Shape, On Purpose**

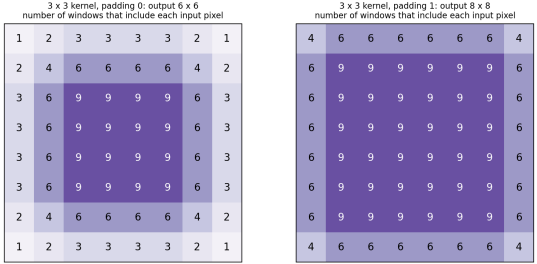
Three Ways to Pad



Source: `F.pad(x, (1, 1, 1, 1), mode=...)` on a 4×4 map; the purple frame is the original map; `scripts/day25_shape_figures.py`.

- **Zeros** (`padding=1`, the default) invents a black border: cheap, and the network learns that the border looks different. **Reflect** mirrors the inside without repeating the edge; **replicate** repeats the edge. Both keep the border looking like the image: `Conv2d(..., padding=1, padding_mode="reflect")`.
- **Same** size needs $p = (k-1)/2$ at stride 1: $p = 1$ for $k = 3$, $p = 2$ for $k = 5$. That is why kernels are odd: an even kernel has no centre pixel and no symmetric padding. `padding="same"` asks PyTorch to compute p for you (stride 1 only).

What the Border Loses Without Padding



Source: window counts of a 3×3 kernel over an 8×8 input, computed with `F.conv_transpose2d` on a map of ones; `scripts/day25_shape_figures.py`.

- Without padding a corner pixel appears in **one** window and a centre pixel in **nine**: the border barely counts, and every layer cuts 2 pixels off each side.
- Five 3×3 layers without padding: $32 \to 30 \to 28 \to 26 \to 24 \to 22$. The map keeps $22^2/32^2 = 47\%$ of its area; the rest fell off the edges.
- With padding 1 every pixel is in at least 4 windows, the size stays 8×8, and the depth of the network no longer depends on the image size. This is why every block in Day 23's recipe writes `padding=1`.

The Formula's Floor, With Numbers

$$n_{\text{out}} = \left\lfloor \frac{n_{\text{in}} + 2p - k}{s} \right\rfloor + 1 \qquad \text{(Day 22; } \lfloor \cdot \rfloor \text{ rounds down)}$$

n_{in}	layer	arithmetic	n_{out}
7	`MaxPool2d(2)` ($k=2, s=2, p=0$)	$\lfloor 5/2 \rfloor + 1 = 2 + 1$	3, last row dropped
7	`Conv2d(k=3, p=1, s=2)`	$\lfloor 6/2 \rfloor + 1 = 3 + 1$	4, nothing dropped
15	`MaxPool2d(2)`	$\lfloor 13/2 \rfloor + 1 = 6 + 1$	7
15	`MaxPool2d(2, ceil_mode=True)`	$\lceil 13/2 \rceil + 1 = 7 + 1$	8, last row kept
75	`MaxPool2d(2)`	$\lfloor 73/2 \rfloor + 1 = 36 + 1$	37
37	`MaxPool2d(2)`	$\lfloor 35/2 \rfloor + 1 = 17 + 1$	18

- On an **odd** size a pool of 2 throws away one row and one column; a stride-2 convolution with padding 1 keeps everything and gives $\lceil n/2 \rceil$. On even sizes both give exactly $n/2$.
- **The 150-pixel chain** (Kaggle's Intel scenes at native size): $150 \to 75 \to 37 \to 18$; the flattened map is $64 \cdot 18 \cdot 18 = 20{,}736$. Dividing 150 by 8 gives 18.75 and a head that does not fit. The 96-pixel chain is $96 \to 48 \to 24 \to 12$: $64 \cdot 144 = 9{,}216$.

Trace the Chain Before You Build the Head

```python
def block(c_in, c_out):
    return nn.Sequential(nn.Conv2d(c_in, c_out, 3, padding=1),
                         nn.BatchNorm2d(c_out), nn.ReLU(), nn.MaxPool2d(2))
features = nn.Sequential(block(3, 16), block(16, 32), block(32, 64))
x = torch.zeros(1, 3, 150, 150)                  # a fake batch of one photo
for name, layer in features.named_children():
    x = layer(x); print(name, tuple(x.shape))
# 0 (1, 16, 75, 75)   1 (1, 32, 37, 37)   2 (1, 64, 18, 18)
print(x.flatten(1).shape)                        # (1, 20736): the Linear's in_features
```

after	96 input	150 input
stage 1 (16 ch)	48×48	75×75
stage 2 (32 ch)	24×24	37×37
stage 3 (64 ch)	12×12	18×18
`Flatten`	9,216	20,736
`Linear(., 6)` parameters	55,302	124,422

- Day 22's habit, made a loop: `named_children()` walks the stages and prints each shape, so the floor is applied by the layers themselves and read off, not computed in your head.
- The head is the only layer whose size depends on the photo size; the table shows how much. Next: what happens when it is wrong, and the two heads that never are.

Shape Debugging: **Reading the Message**

The Method: Both Sides Are Named

Rule

Every shape error names **two tensors**: the layer's and yours. Find the word that says which is which, then compare the numbers position by position: batch, channels, height, width.

the message says	it means	the layer
`mat1 and mat2 ... (32x20736 and 9216x6)`	`mat1` is your flattened batch, `mat2` the weight	`Linear(9216, 6)`
`weight of size [16, 3, 3, 3]`	16 kernels over 3 channels, 3×3	`Conv2d(3, 16, 3)`
`expected input[32, 150, 150, 3]`	your batch, as the layer received it	the line before the layer
`Given input size: (16x1x1)`	your map, channels then height then width	`MaxPool2d`
`target batch_size (16)`	your labels	`CrossEntropyLoss`

- The error is **raised** in the layer and **caused** earlier, usually in the Dataset or the head. Fix the cause: a `Linear` that fails because the batch arrived as $(32, 150, 150, 3)$ is not a `Linear` problem.
- Reproduce it in one line with a zero batch of the shape the message shows, fix, rerun that line, then the loop. Every message on the next two frames was produced that way with PyTorch 2.11.

Errors Inside the Network

message (verbatim)	cause	fix
`mat1 and mat2 shapes cannot be multiplied (32x20736 and 9216x6)`	the head was sized for another image size: the batch flattens to 20,736, the `Linear` expects 9,216	`Linear(20736, k)`; better, a head that does not depend on the size (next frame)
`Given groups=1, weight of size [16, 3, 3, 3], expected input[32, 150, 150, 3] to have 3 channels, but got 150 channels instead`	the batch is height, width, channels (NumPy or PIL order); PyTorch wants channels first	`x.permute(0, 3, 1, 2)`, or `ToTensor()` in the Dataset, which does it
`... weight of size [16, 3, 3, 3], expected input[32, 1, 96, 96] to have 3 channels, but got 1 channels instead`	grey photos into a colour network	`.convert("RGB")` when opening, or `Conv2d(1, 16, 3)`
`... weight of size [16, 3, 3, 3], expected input[1, 32, 150, 150] to have 3 channels, but got 32 channels instead`	a 3-D batch (no channel axis): PyTorch read it as one image of 32 channels	`x.unsqueeze(1)` for grey, or load the colour channels
`expected 4D input (got 3D input)` (from `BatchNorm2d`)	one image sent without a batch axis; `Conv2d` alone would have accepted it silently	`model(img[None])`
`Input type (unsigned char) and bias type (float) should be the same`	pixels still `uint8`, 0 to 255	`x.float() / 255`, or `ToTensor()`
`Calculated padded input size per channel: (2 x 2). Kernel size: (3 x 3). Kernel size can't be greater than actual input size`	too many poolings for the image: the map shrank below the kernel	fewer stages, or larger photos; count the chain first

Errors at the Loss, and the Ones That Stay Silent

message (verbatim)	cause	fix
`0D or 1D target tensor expected, multi-target not supported`	labels of shape $(32, 1)$, read from a column	`y.squeeze(1)`, or index the column as `df["label"].values`
`expected scalar type Long but found Float`	labels stored as floats	`y.long()`; in the Dataset `torch.tensor(label, dtype=torch.int64)`
`Target 6 is out of bounds.`	classes numbered 1 to 6 with a head of 6 outputs, or a class missing from `class_to_idx`	number classes 0 to $k-1$; `Linear(., k)` with k the number of distinct labels
`Expected input batch_size (32) to match target batch_size (16).`	images and labels from different loaders or a wrong `collate`	one Dataset that returns `(image, label)` pairs

No message at all

- `CrossEntropyLoss(logits, y)` with `y` a **float** tensor of shape $(32, 6)$ runs: PyTorch reads it as class probabilities. A one-hot made by mistake, or a wrong tensor of the right shape, trains without a word.
- `Conv2d` accepts a single image of shape $(3, 96, 96)$ and returns $(16, 96, 96)$: the missing batch axis only surfaces at the next layer, with a message about a different tensor.
- A `Linear` that happens to fit a wrong flatten, because two different maps have the same product, trains on scrambled features. The shape trace of the last frame catches all three.

The Tool: `torchinfo.summary`

```
from torchinfo import summary                    # pip install torchinfo
summary(SceneNet(), input_size=(1, 3, 96, 96),
        col_names=("output_size", "num_params"), depth=2)
# Layer (type:depth-idx)                  Output Shape          Param #
# SceneNet                                [1, 6]                --
# |-Sequential: 1-1                       [1, 64, 12, 12]       --
# |    |-Sequential: 2-1                  [1, 16, 48, 48]       480
# |    |-Sequential: 2-2                  [1, 32, 24, 24]       4,704
# |    |-Sequential: 2-3                  [1, 64, 12, 12]       18,624
# |-Sequential: 1-2                       [1, 6]                --
# |    |-Flatten: 2-4                     [1, 9216]             --
# |    |-Linear: 2-5                      [1, 6]                55,302
# Total params: 79,110    Forward/backward pass size (MB): 4.13
```

- One call runs a zero batch through the model and prints every shape and parameter count: the trace loop of the last section, for any model, in one line. Run it before the first epoch and again after every change to the architecture.
- Read it top to bottom like the chain table: channels up, map down, one big `Linear` at the end holding 70% of the parameters. The memory line tells you whether the batch size will fit the GPU.

Heads That Never Break: LazyLinear and Global Average Pooling

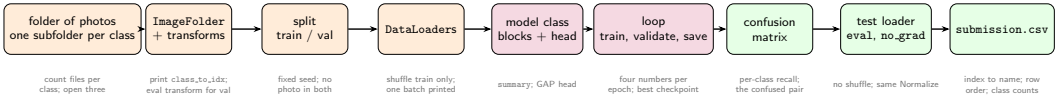
```
head = nn.Sequential(nn.Flatten(), nn.LazyLinear(6))   # 1. PyTorch measures the size
head(torch.zeros(2, 64, 18, 18)); print(head[1].in_features)   # first batch: 20736
head = nn.Sequential(nn.AdaptiveAvgPool2d(1), nn.Flatten(), nn.Linear(64, 6))  # 2. GAP
net = nn.Sequential(features, head)
print(net(torch.zeros(2, 3, 96, 96)).shape, net(torch.zeros(2, 3, 150, 150)).shape)
```

parameters	96 input	150 input
`Flatten` + `Linear`	79,110	148,230
GAP + `Linear(64, 6)`	24,198	24,198

- `LazyLinear` fills in `in_features` from the first batch it sees: the number is still tied to the photo size, but you never type it. Run one batch through before creating the optimizer.
- Global average pooling removes the dependence: the head reads 64 numbers whatever the map size, with 390 parameters instead of 55,302, and one trained model serves 96 and 150 pixel photos alike (Day 23; Lin et al., 2014).

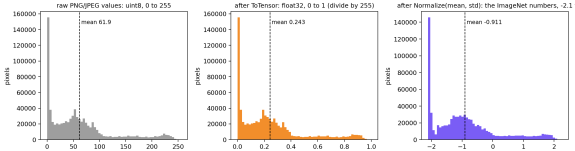
Image Classification: **From Folder to Submission**

The Pipeline, Stage by Stage



- Nine stages, and under each the check that costs a minute. Weeks 2 and 3 built the right-hand half; Days 22 and 23 built the model; this deck is about the stages where image runs go wrong: the transforms, the split, the head, and the hand-in.
- Friday's task and every Kaggle classification task has this shape. The checks are what separate a notebook that scores from one that "ran fine".

What the Transforms Do to a Pixel, With Numbers



Source: all pixel values of scikit-learn's `flower.jpg` (three channels) before and after `ToTensor` and `Normalize` with the ImageNet means and standard deviations; `scripts/day25_shape_figures.py`.

- `ToTensor`: `uint8` $(H, W, 3)$ becomes `float32` $(3, H, W)$ divided by 255: a red value of 200 becomes 0.784.
- `Normalize(mean, std)`: per channel, $(x - \mu)/\sigma$. With the ImageNet numbers $\mu_R = 0.485$, $\sigma_R = 0.229$: $(0.784 - 0.485)/0.229 = 1.307$; black becomes -2.118, white 2.249. Inputs centred near 0 with spread near 1 are what Day 17 asked of every layer's input, done here by hand for the first one.
- ImageNet numbers with a pretrained network (Week 5), the **training split's own** otherwise; validation and test always get the **same** numbers, or accuracy drops silently.

In Code: Folder to Loaders

```
MEAN, STD = (0.485, 0.456, 0.406), (0.229, 0.224, 0.225)
norm = [transforms.ToTensor(), transforms.Normalize(MEAN, STD)]
tf_train = transforms.Compose([transforms.RandomResizedCrop(96, scale=(0.8, 1.0)),
                               transforms.RandomHorizontalFlip()] + norm)
tf_eval = transforms.Compose([transforms.Resize((96, 96))] + norm)
root = f"{path}/seg_train/seg_train"              # one subfolder per class
idx = torch.randperm(14034, generator=torch.Generator().manual_seed(0))
train_ds = Subset(datasets.ImageFolder(root, tf_train), idx[2800:])  # two ImageFolders
val_ds = Subset(datasets.ImageFolder(root, tf_eval), idx[:2800])     # two transforms
print(train_ds.dataset.class_to_idx)     # {'buildings': 0, 'forest': 1, ...}
train_loader = DataLoader(train_ds, batch_size=64, shuffle=True, num_workers=2)
val_loader = DataLoader(val_ds, batch_size=64, shuffle=False, num_workers=2)
xb, yb = next(iter(train_loader)); print(xb.shape, yb[:6])    # (64, 3, 96, 96)
```

- **Two** `ImageFolder` **objects** over the same folder, one per transform, split by one random permutation. One object cut by `random_split` shares its transform, so the validation photos get augmented too.
- The last line is the Day 19 habit: one batch's shape, dtype and labels, printed before the model exists. Kaggle's Intel set: 14,034 training photos in six folders.

The Class-Index Trap

Where it breaks

`ImageFolder` numbers the classes in **alphabetical order** of the folder names. The competition's sample submission may want names, or its own ids in another order. A model that is 90% right on validation hands in 90% wrong rows if the mapping is guessed.

- Build every mapping from data you printed: `class_to_idx` from the training folder, the id table from the competition's CSV. Never type a class list by hand.
- When labels come from a CSV instead of folders, the same rule: `sorted(df.label.unique())` is your index order; store it and invert it.

```
c2i = train_ds.dataset.class_to_idx
i2c = {i: c for c, i in c2i.items()}
# i2c[2] == 'glacier', i2c[5] == 'street'

logits = model(xb)              # (64, 6)
pred = logits.argmax(dim=1)     # (64,) in
    0..5
names = [i2c[int(i)] for i in pred]

# the competition's own ids:
ids = pd.read_csv("classes.csv")  # name,
    id
name2id = dict(zip(ids.name, ids.id))
rows = [name2id[n] for n in names]
```

Check: `pd.Series(names).value_counts()` on the test predictions. Six classes of roughly equal size in the data and 80% of the predictions in one class means the mapping or the transform is wrong, not the model.

Reading the Result: Confusion Matrix, Per-Class Recall, Macro F1

true ↓ pred. →	glacier	mountain	sea
glacier (10)	**7**	3	0
mountain (10)	2	**8**	0
sea (10)	0	1	**9**

A worked example: 30 validation photos, ten per class; the numbers are chosen for the arithmetic, not measured.

- **Accuracy** $= (7 + 8 + 9)/30 = 0.80$.
- **Recall** per class, the diagonal over the row: 0.7, 0.8, 0.9. **Precision**, the diagonal over the column: $7/9 = 0.78$, $8/12 = 0.67$, $9/9 = 1.0$.
- **F1** per class $= 2 \cdot P \cdot R/(P + R)$: 0.74, 0.73, 0.95; **macro F1** is their mean, 0.80. Day 09's metric for unbalanced classes; Kaggle tasks name it in the rules.
- The matrix says more than the accuracy: five of the six errors are glacier against mountain. That is the pair to look at: open the confused photos, and expect both classes to need more epochs, more augmentation or a larger photo size, not the whole model.
- In `sklearn.metrics`, `confusion_matrix` and `classification_report` print all of this from two lists; collect `y_true` and `y_pred` in the validation pass, on the CPU, as integers.

Test Time: The Checklist

```
test_ds = TestFolder(f"{path}/seg_pred/seg_pred", tf_eval)  # Day 19: (image, file name)
test_loader = DataLoader(test_ds, batch_size=64, shuffle=False)
model.load_state_dict(torch.load("best.pt")); model.eval()
rows = []
with torch.no_grad():
    for xb, names in test_loader:
        pred = model(xb.to(device)).argmax(1).cpu()
        rows += [(n, i2c[int(p)]) for n, p in zip(names, pred)]
sub = pd.DataFrame(rows, columns=["image", "label"])
sample = pd.read_csv("sample_submission.csv")
sub = sub.set_index("image").loc[sample.image].reset_index()   # the sample's order
sub.to_csv("submission.csv", index=False)
```

Before you press submit

1. `tf_eval`, never `tf_train`: no crops or flips, the same `Normalize`. 2. `model.eval()`, `torch.no_grad()`, the **best** checkpoint.
3. `shuffle=False`; the file name next to every prediction; the rows in the sample's order.
4. Index to **name** through the printed mapping; `sub.label.value_counts()` looks like the training split; `sub.shape` equals the sample's.

Shapes: **Olympiad Practice**

Practice Task: Six Kinds of Garbage

Task

Kaggle's *Garbage Classification* set (Day 22's Lab 1): 2,527 photos of 512 × 384 pixels in six folders, `cardboard` (403), `glass` (501), `metal` (410), `paper` (594), `plastic` (482), `trash` (137). A hidden test folder, scored by **macro F1**; hand in `image,label` with the class *name*.

- **Question 1:** you keep the 4 : 3 shape, resize to 192 × 144 (width, height) and run three blocks of `Conv2d` (3 × 3, padding 1), `BatchNorm2d`, ReLU, `MaxPool2d(2)` with 16, 32, 64 channels. The map after each block, the `in_features` after `Flatten`, and what a global-average-pooling head changes?
- **Question 2:** you reuse the Day 22 head and the run stops with `mat1 and mat2 shapes cannot be multiplied (64x27648 and 9216x6)`. What happened, and which of the two fixes do you choose?
- **Question 3:** `trash` has 137 photos against 594 for `paper`. Which number chooses the epoch, which loss argument helps, and what do you check in the submission before uploading?

Source: Kaggle dataset `asdasdasasdas/garbage-classification` (the TrashNet photos of Yang and Thung, 2016); names, counts and size are the dataset's own.

Solution Idea and Code

1. Height $144 \to 72 \to 36 \to 18$, width $192 \to 96 \to 48 \to 24$ (even sizes, no floor loss): maps $(16, 72, 96)$, $(32, 36, 48)$, $(64, 18, 24)$; `in_features` $= 64 \cdot 18 \cdot 24 = 27{,}648$. With `AdaptiveAvgPool2d(1)` the head reads 64 numbers whatever the photo size: `Linear(64, 6)`, 390 parameters, and the same model would accept the 512×384 originals.
2. `mat1` is your batch of 64 photos flattened to 27,648; `mat2` is a weight for 9,216, the 96×96 head of Day 22. Either `Linear(27648, 6)` (165,894 parameters, tied to 192×144) or the GAP head; choose GAP.
3. Choose the epoch on validation **macro F1**, not accuracy: a model that never predicts `trash` still scores 0.95 accuracy on the other five. `CrossEntropyLoss(weight=w)` with $w_c = N/(6 \cdot n_c)$ gives `trash` weight $2{,}527/822 = 3.07$ and paper 0.71. Before uploading: `sub.label.value_counts()` should show all six names with `trash` the rarest, and the names must come from `class_to_idx` inverted, not typed.

```
counts = torch.tensor([403, 501, 410, 594, 482, 137.])   # alphabetical class order
w = counts.sum() / (6 * counts)          # [1.05, 0.84, 1.03, 0.71, 0.87, 3.07]
loss_fn = nn.CrossEntropyLoss(weight=w.to(device))
head = nn.Sequential(nn.AdaptiveAvgPool2d(1), nn.Flatten(), nn.Linear(64, 6))
```

Shapes: **Check Yourself**

Check Yourself

Answer before turning the page.

1. **Kernels.** $K = \begin{pmatrix} 1 & 1 & 1 \\ 1 & -8 & 1 \\ 1 & 1 & 1 \end{pmatrix}$: its output on a flat patch of 90s, and on 90s with a 200 in the centre? Which family, and what does adding 1 to the centre weight make it?
2. **Shapes.** A 150×150 colour photo through `Conv2d(3, 32, 5)` (no padding), `MaxPool2d(2)`, then `Conv2d(32, 32, 3, padding=1, stride=2)`: the three output shapes, the `in_features` after `Flatten`, and the first convolution's parameter count.
3. **Errors.** `weight of size [16, 3, 3, 3], expected input[16, 224, 224, 3] to have 3 channels, but got 224 channels instead`: which tensor is yours, what is wrong with it, where is the fix?
4. **Pipeline.** Validation accuracy 0.91 on six balanced classes; the submitted file scores 0.17. Three places to look, in order, with a one-line test for each.

Answers

1. The weights sum to $8 - 8 = 0$, so a flat patch gives $90 \cdot 0 = 0$: an edge family kernel (an eight-neighbour Laplacian). With 200 in the centre: $8 \cdot 90 - 8 \cdot 200 = 720 - 1600 = -880$, a strong response to the dot. Adding 1 to the centre makes the sum 1: identity minus this Laplacian, a sharpen kernel.
2. `Conv2d(3, 32, 5)`: $150 - 5 + 1 = 146$, shape $(32, 146, 146)$. Pool: $\lfloor 144/2 \rfloor + 1 = 73$, shape $(32, 73, 73)$. Stride-2 conv with padding 1: $\lfloor (73 + 2 - 3)/2 \rfloor + 1 = 36 + 1 = 37$, shape $(32, 37, 37)$. `in_features` $= 32 \cdot 37 \cdot 37 = 43{,}808$. The first convolution has $32 \cdot (3 \cdot 5 \cdot 5) + 32 = 2{,}432$ parameters.
3. `input[16, 224, 224, 3]` is yours: a batch of 16 photos stored height, width, channels, so PyTorch took 224 as the channel count. The layer is right; the cause is the Dataset returning a NumPy or PIL array without `ToTensor` (or `permute(2, 0, 1)`). Fix `__getitem__`, then print one batch: it must be $(16, 3, 224, 224)$.
4. (a) The test transform: print `test_ds.transform`; it must be `tf_eval` with the same `Normalize`. (b) The order: the loader must have `shuffle=False` and the rows must be reordered to `sample_submission`; check `(sub.image == sample.image).all()`. (c) The mapping: `sub.label.value_counts()` should look like six classes of similar size; one dominant class or names that do not match the sample's means the index-to-name step is wrong. 0.17 is one in six: the predictions are no better than chance after the file was built, so the model is not the first suspect.

Shapes: **Limitations & What Comes Next**

Limitations

- **The formula covers the layers we use.** Square kernels, one padding on each side, no dilation, no groups. Those two arguments exist in `Conv2d`, change the arithmetic, and are named here only so that the documentation does not surprise you.
- **The messages are PyTorch 2.11's.** Their wording changes between versions; the structure, two tensors named and compared, does not. Read for the shapes, not for the sentence.
- `summary` **counts, it does not time.** Models with equal parameter counts can differ ten times in speed; the mult-adds line is the better guide, one timed epoch the real one.
- **Everything here trains from scratch on small photos.** From Week 5 on, the backbone is pretrained and only the head is yours; the pipeline, the transforms with the ImageNet numbers and every check on these frames stay the same.

The common mistake

Fixing the error where it is *raised*. A `Linear` that fails, a convolution that complains about 150 channels and a loss that wants `Long` are all caused earlier, in the Dataset, the transform or the head's arithmetic. Reproduce with a zero batch, fix the cause, print the shapes again.

What Comes Next

- **Day 26, self-study:** YOLO's output tensor and loss, U-Net's blocks, and IoU, mAP and Dice computed by hand; every shape there follows the formula of this deck.
- **Day 27, Olympiad practice:** the vision habits that win points: augmentation that moves boxes and masks along with the photo, test-time augmentation, the checks of this deck as a routine.
- **Friday:** the Week 4 competition.
- **Week 5:** pretrained backbones: the same pipeline, the same `Normalize` numbers, a head you replace and a backbone you freeze or fine-tune.

Shapes: **Summary**

Summary

- **Kernels:** the sum tells the family (1 keeps a picture, 0 keeps only change); a weight $[a, b, k, k]$ is a kernels over b channels; a 1×1 kernel is a `Linear` across channels; stacked 3×3 kernels see $r_\ell = r_{\ell-1} + (k-1)j_{\ell-1}$: $3, 5, 10, 14, 24$.
- **Padding and stride:** zeros, reflect or replicate; $p = (k-1)/2$ keeps the size (why kernels are odd); without padding the corner is seen once and every layer loses 2 pixels; the formula rounds *down*: $150 \to 75 \to 37 \to 18$, a head of 20,736.
- **Shape errors:** two tensors are named; find yours, compare position by position, fix the cause (Dataset, transform, head). Float targets and unbatched images fail silently.
- **Tools:** the trace loop, `torchinfo.summary`, `LazyLinear` for the number you never type, global average pooling for a head that fits any photo size.
- **The pipeline:** two `ImageFolders` with two transforms, a seeded split, loaders (shuffle the training one only), four numbers per epoch and the best checkpoint, the confusion matrix and its confused pair.
- **Test time:** `tf_eval`, `eval` and `no_grad`, no shuffle, index to name from the printed mapping, rows in the sample's order, label counts checked.

Shapes: **References & Resources**

References & Resources

[1] Dumoulin, V. and Visin, F. (2016). A guide to convolution arithmetic for deep learning. arXiv:1603.07285. The output-shape formula, padding, stride and the transposed convolution, with pictures; the reference for every shape on these frames.

[2] LeCun, Y., Bottou, L., Bengio, Y. and Haffner, P. (1998). Gradient-based learning applied to document recognition. *Proceedings of the IEEE* 86(11). Local receptive fields, shared weights and subsampling.

[3] Krizhevsky, A., Sutskever, I. and Hinton, G. (2012). ImageNet classification with deep convolutional neural networks. *NeurIPS* 25. Fig. 3: the learned first-layer kernels that look like the hand-made gallery.

[4] Simonyan, K. and Zisserman, A. (2015). Very deep convolutional networks for large-scale image recognition. *ICLR*. Two 3×3 layers in place of one 5×5: the receptive-field argument of Section 2.3.

[5] Lin, M., Chen, Q. and Yan, S. (2014). Network in network. *ICLR*. Global average pooling and the 1×1 convolution as a per-pixel linear layer.

[6] Sobel, I. and Feldman, G. (1968). A 3×3 isotropic gradient operator for image processing. Talk at the Stanford Artificial Intelligence Project; the edge kernels of the gallery.

[7] PyTorch documentation: `nn.Conv2d` (`padding`, `padding_mode`, `padding="same"`), `nn.MaxPool2d` (`ceil_mode`), `nn.LazyLinear`, `nn.AdaptiveAvgPool2d`, `nn.CrossEntropyLoss` (class indices or class probabilities as targets). https://pytorch.org/docs/stable/nn.html. Every error message quoted was produced with PyTorch 2.11.

References & Resources (cont.)

[8] torchvision documentation: `datasets.ImageFolder` (`class_to_idx`, sorted class names), `transforms.ToTensor`, `transforms.Normalize`, `transforms.RandomResizedCrop`. https://pytorch.org/vision/stable/

[9] Yep, T. *torchinfo*: model summaries for PyTorch. https://github.com/TylerYep/torchinfo. The summary on the tools frame is its output for the deck's `SceneNet` at 96×96.

[10] The ImageNet channel means $(0.485, 0.456, 0.406)$ and standard deviations $(0.229, 0.224, 0.225)$ used by every pretrained torchvision model: https://pytorch.org/vision/stable/models.html.

[11] Kaggle dataset `puneet6060/intel-image-classification`: 14,034 training photos of 150×150 pixels in six folders, 3,000 labelled test photos, 7,301 unlabelled prediction photos; the class names, counts and sizes quoted are the dataset's own. https://www.kaggle.com/datasets/puneet6060/intel-image-classification

[12] Figures without another source: `slides-builder/scripts/day25_shape_figures.py` (kernel gallery, padding modes, window counts and pixel scaling, all computed with `torch.nn.functional` on scikit-learn's sample photograph `flower.jpg`). No model was trained for this deck; the confusion matrix is a worked example and says so.